CSCI 410
Elements of Computing Systems

13 – The Hack Virtual Machine

Part 2: Program Flow and Function Calls

CS@Mines

ANNOUNCEMENTS

Thursday, March 19

- 100% Workday for Project 7 and 8

Project 7 & 8: Due Sunday April 5th.

QUESTIONS?
COMMENTS?
CONCERNS?

Lecture Outline

- Introductory stuff
- Flow control
  - VM
  - Implementation
- Functions and function calls
  - VM
  - Implementation
- Putting it all together
- Concluding remarks

Hack VM Overview

Project 7

Arithmetic/logical operators:

- add
- sub
- neg
- eq
- gt
- lt
- and
- or
- not

Memory access commands:

- pop *segment i*
- push *segment i*

Project 8

Branching commands:

- label *label*
- goto *label*
- if-goto *label*

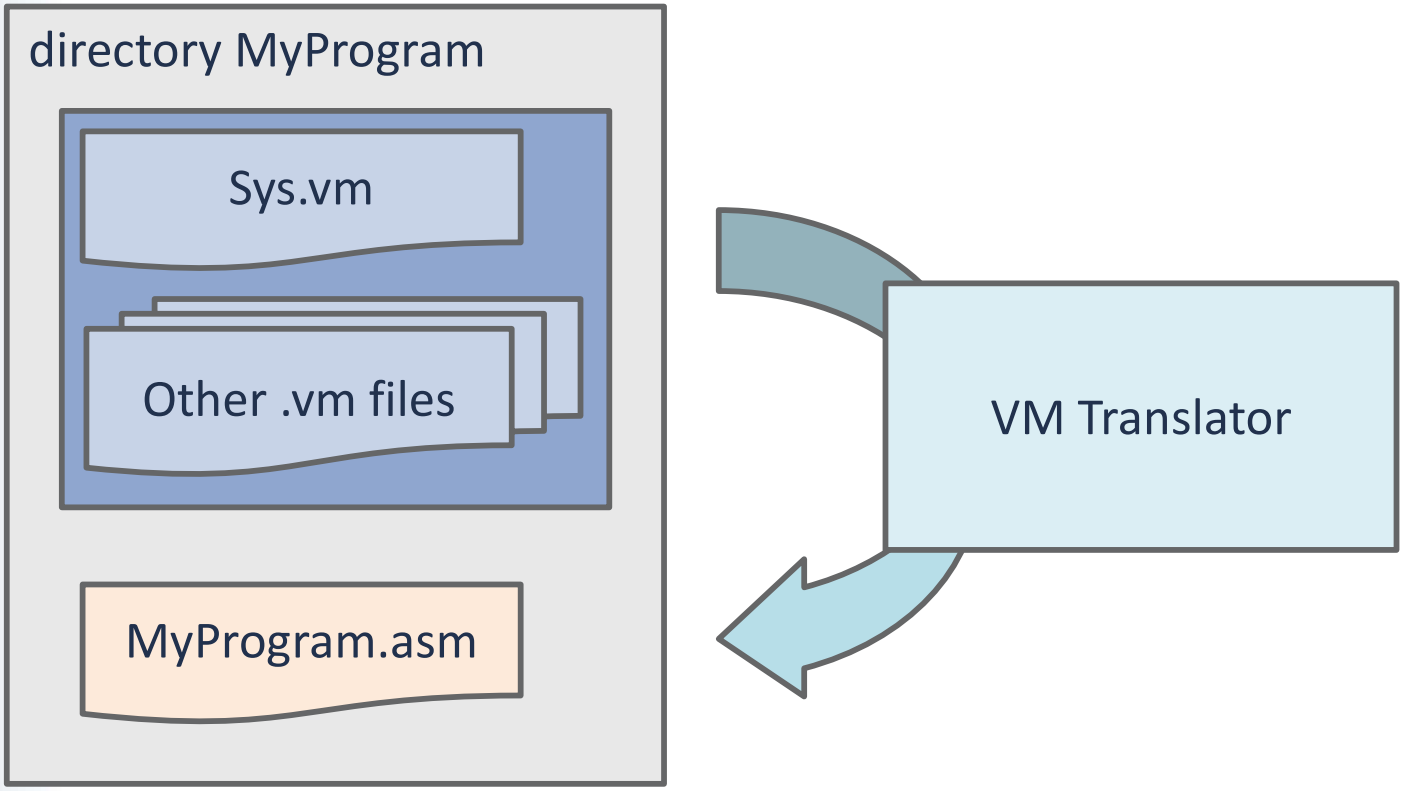
Function commands:

- function *name varCount*
- call *name argCount*
- return

Project 8 Big Picture

- A directory will contain a Hack VM program
  - Individual .vm files implementing modules
  - Each .vm file only contains *function* definitions
  - One special file is Sys.vm with function Sys.init
- Your translator:
  - Scan directory for .vm files
  - Create one .asm output file in the directory (named for the directory) – this will be the single output
  - Output *bootstrap* code
  - Output translations of each .vm file in turn

Big Picture Picture



Implementing VM on Hack Platform

The challenge:

- Done:
  - Memory ops (push from/pop to segments)
  - Arithmetic, Boolean operations
- To do:
  - Manage program flow with goto, if-goto and label
    - (Almost) direct translation to assembly code
    - Just have to discuss how to translate labels
  - Make function call protocol work on Hack platform
  - Add in initialization code

Program Flow and Function Calls

Key constructs in modern languages:

- Flow control (conditionals/branching)
- Function calls (subroutines, procedures, etc.)

Example code (high-level):

```
// compute roots of quadratic equation
if (a != 0) {
    x1 = (-b + sqrt(b * b – 4 * a * c)) / (2 * a);
    x2 = (-b - sqrt(b * b – 4 * a * c)) / (2 * a);
} else {
    x1 = x2 = -c / b;
}
```

PROGRAM FLOW CONTROL

Program Flow

Example code (high-level):

```
// compute roots of quadratic equation
if (a != 0) {
    x1 = (-b + sqrt(b * b – 4 * a * c)) / (2 * a);
    x2 = (-b - sqrt(b * b – 4 * a * c)) / (2 * a);
} else {
    x1 = x2 = -c / b;
}
```

Also, while/for loops, goto (when not considered harmful)

Flow Control in the Hack VM

```
function mult 1
    push constant 0
    pop local 0
  label loop
    push argument 0
    push constant 0
    eq
    if-goto end
    push argument 0
    push 1
    sub
    pop argument 0
    push argument 1
    push local 0
    add
    pop local 0
    goto loop
  label end
    push local 0
    return
```

`label` *name*

- Label a location **in the current function**
  - Ultimately, all VM code will live in functions – once we implement support for functions!
- Must be unique within the function

`goto` *name*

- Unconditionally jump to label *name*

`if-goto` *name*

- Pop top value on stack
- If not zero, jump to label *name*

Implementing `label`

label *name*

- The `label` VM command defines a unique location in a *function's* code
- This has to be translated into a *globally* unique location in assembly code
- Solution: use assembly label `function-name$label-name`
  - Function names *are unique globally*
  - VM labels are unique *per function*
  - Thus the above construct will be unique in the assembly program

Since all VM code resides in functions, VM translator just needs to keep track of current function name.

Implementing `goto`, `if-goto`

goto *name*
if-goto *name*

`goto`: unconditional jump

`if-goto`: jump on *not-equal to zero*

FUNCTIONS AND FUNCTION CALLS

Functions

Example code (high-level):

```
// compute roots of quadratic equation
if (a != 0) {
    x1 = (-b + sqrt(b * b – 4 * a * c)) / (2 * a);
    x2 = (-b - sqrt(b * b – 4 * a * c)) / (2 * a);
} else {
    x1 = x2 = -c / b;
}
```

In principle:

- Functions extend the language with user-defined operations
- Functions should behave like built-in operations
  - In VM, e.g., should pop arguments from stack, push result on stack

Functions in the Hack VM

- `function` *`name nLocals`*
  - Define the function named *name*
  - *name* is global scope, must be unique in that scope
  - *nLocals* specifies how many **local** variables are needed
  - Any function can call any other function in VM
- `call` *`name nArgs`*
  - Call the function named *name*
  - *nArgs* is the number of **arguments** being passed
    - arguments must have been pushed onto stack
- `return`
  - Returns from a function
    - Arguments will have been popped from stack
    - Return value must have been pushed onto stack

LOCAL/ARGUMENT

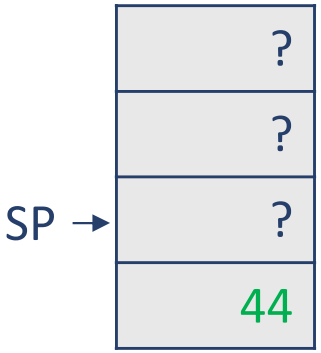
- LOCAL
  - Name: LCL
  - Where: MEM[1]
  - What: The address of the LOCAL segment.
- ARGUMENT
  - Name: ARG
  - Where: MEM[2]
  - What: The address of the ARGUMENT segment.

VM Functions, Illustrated

```
…
  // compute 4 * 11
  push constant 4
  push constant 11
  call mult 2
…
```

Before call

After call




```
function mult 1
    push argument 0
    pop local 0
  label loop
    push argument 0
    push constant 0
    eq
    if-goto end
    push argument 0
    push 1
    sub
    pop argument 0
    push argument 1
    push local 0
    add
    pop local 0
    goto loop
  label end
    push local 0
    return
```

VM Functions, Illustrated

```
…
  // compute 4 * 11
  push constant 4
  push constant 11
  call mult 2
…
```

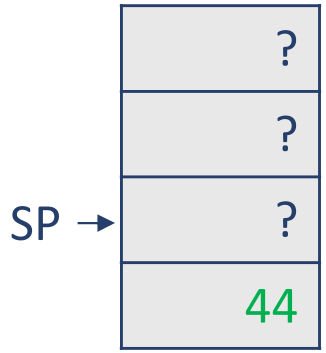
2 arguments

```
function mult 1
    push argument 0
    pop local 0
  label loop
    push argument 0
    push constant 0
    eq
    if-goto end
    push argument 0
    push 1
    sub
    pop argument 0
    push argument 1
    push local 0
    add
    pop local 0
    goto loop
  label end
    push local 0
    return
```

size 1 local segment

Before call

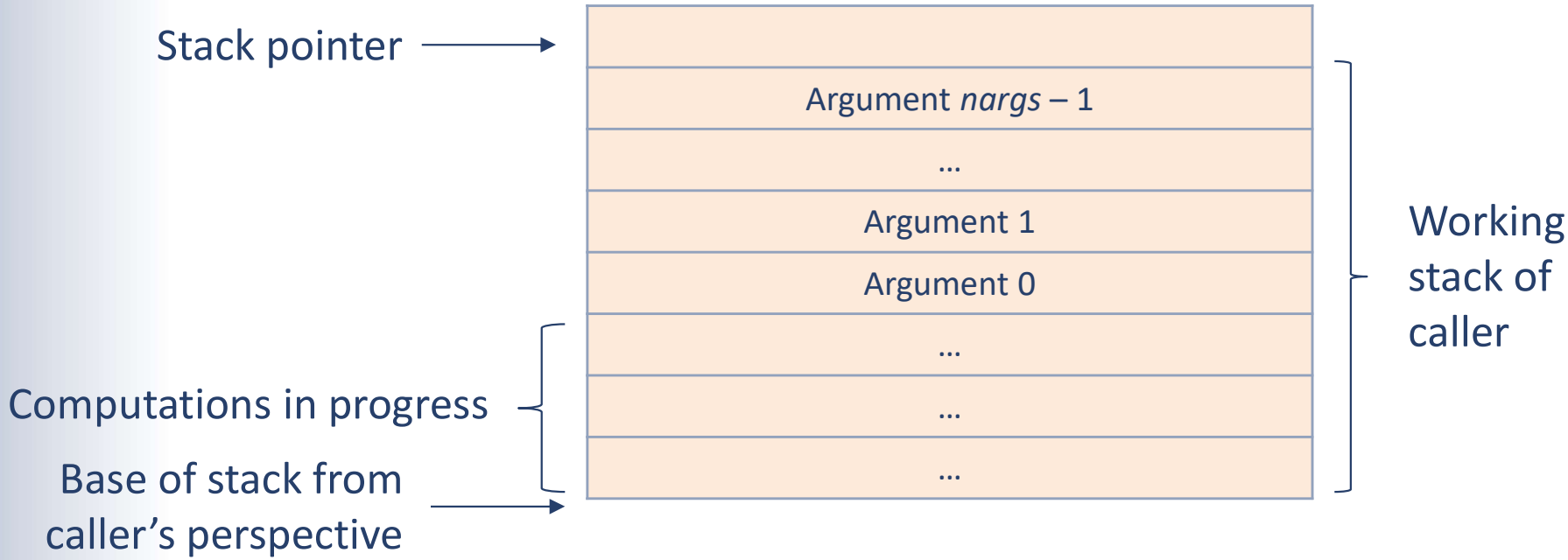
After call



Function Call: Caller's View of Stack (Before)

```
call name nargs
```

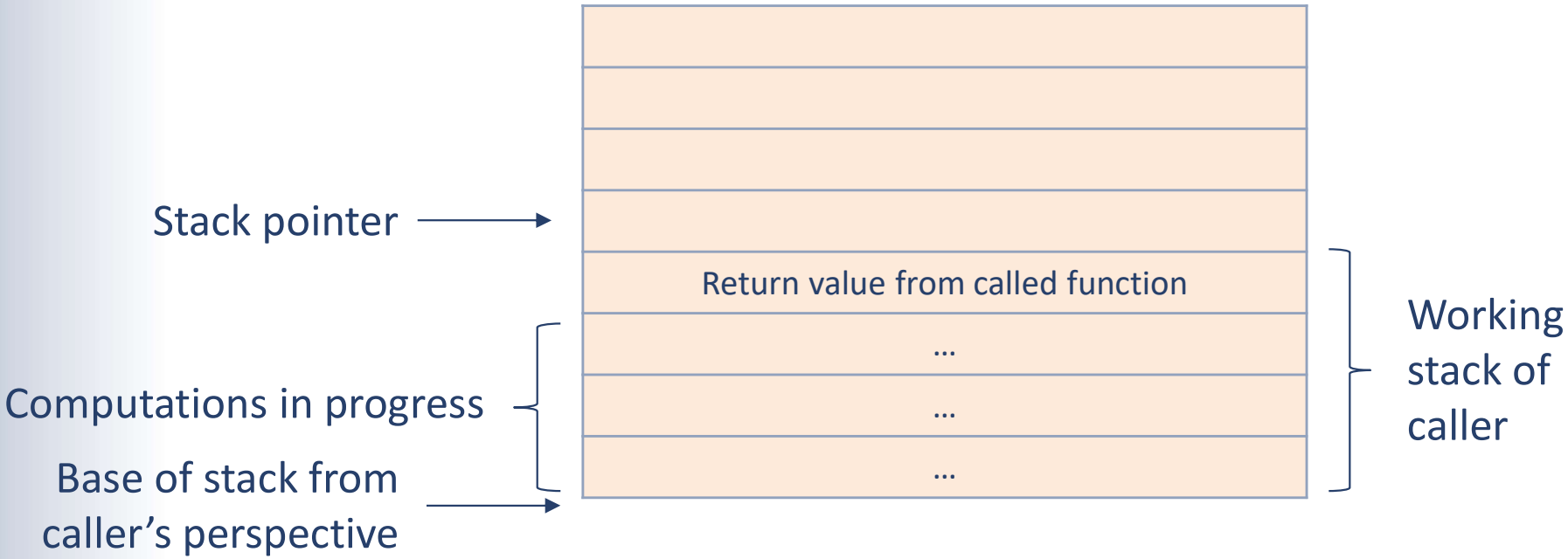
Arguments to called function have been pushed:



Function Call: Caller's View of Stack (After)

```
call name nargs
```

Arguments have been popped, return value is on top of stack. Computations in progress are undisturbed.



Caller's Memory Environment (Before and After Call)

Caller has the following pointers and segments:

- ARG points to `argument` segment
- LCL points to `local` segment
- THIS points to `this` segment
- THAT points to `that` segment

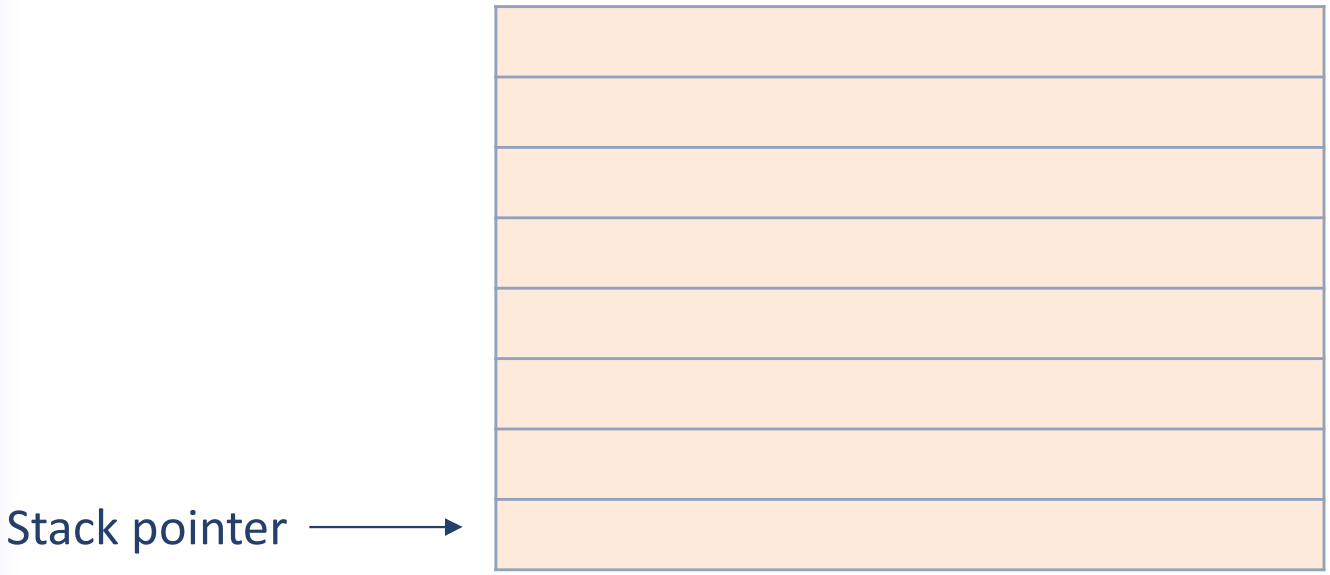
Pointers & segment contents are same before and after.

The only change is to the stack.

Function Call: Callee's View of Stack (Start of Call)

```
function name nlocals
```

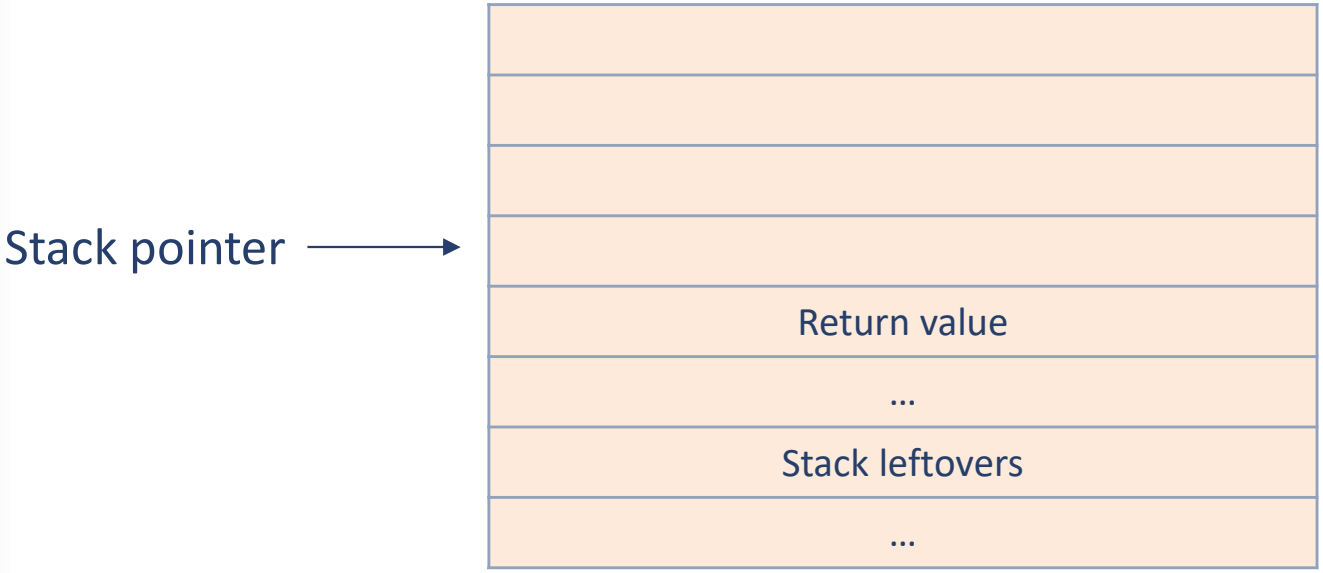
Working stack is empty (note that arguments are in the `argument` segment, not on the stack, from callee's point of view).



Function Call: Callee's View of Stack (Before `return`)

```
function name nlocals
```

Return value is on top of the stack. The stack *may* contain other leftovers from computation, which will be ignored.



Callee's Environment (Start of Call)

```
function name nlocals
```

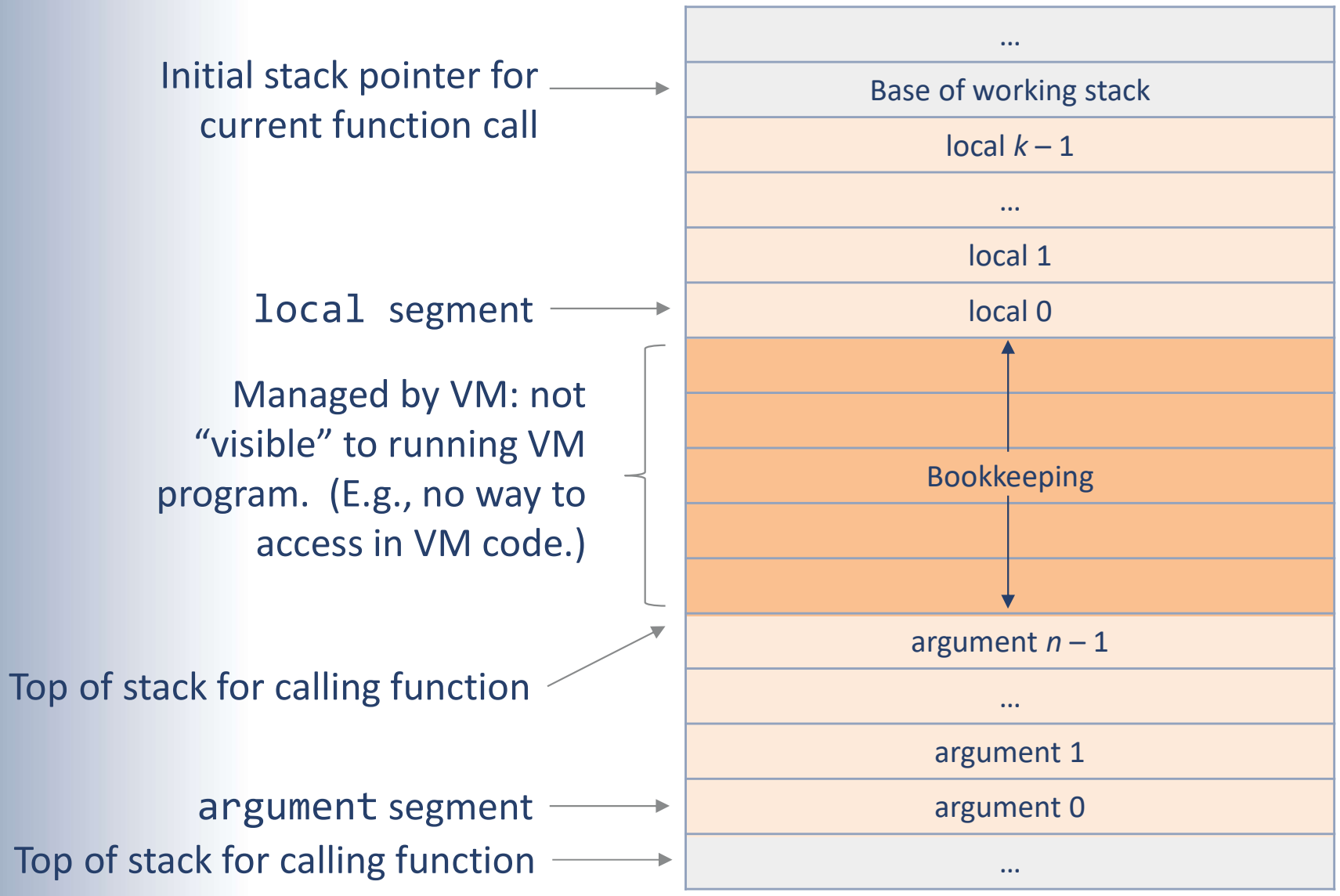
Callee has the following pointers and segments:

- ARG points to `argument` segment
- LCL points to `local` segment
  - `local` has *`nlocals`* entries
  - Contains 0 values in all entries at start of call
- THIS points to `this` segment
- THAT points to `that` segment

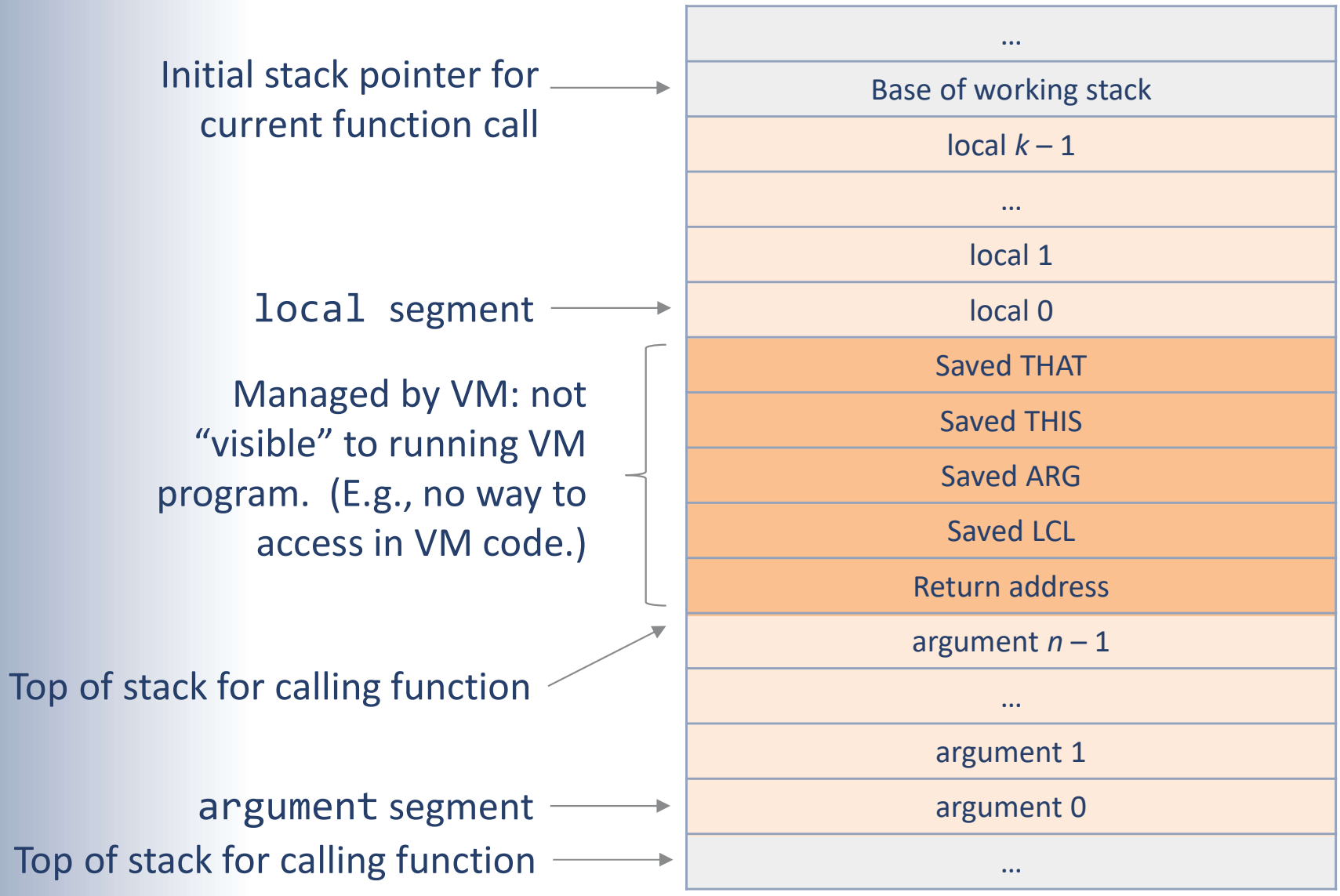
Implementation

- Previous slides describe things from *VM's* point of view
- Now, we turn to how functions are mapped onto the Hack hardware platform

Stack Frame


Initial stack pointer for current function call
local segment
Managed by VM: not "visible" to running VM program. (E.g., no way to access in VM code.)
Top of stack for calling function
argument segment
Top of stack for calling function
...
Base of working stack
local k – 1
...
local 1
local 0
Bookkeeping
argument n – 1
...
argument 1
argument 0
...

Stack Frame Detailed



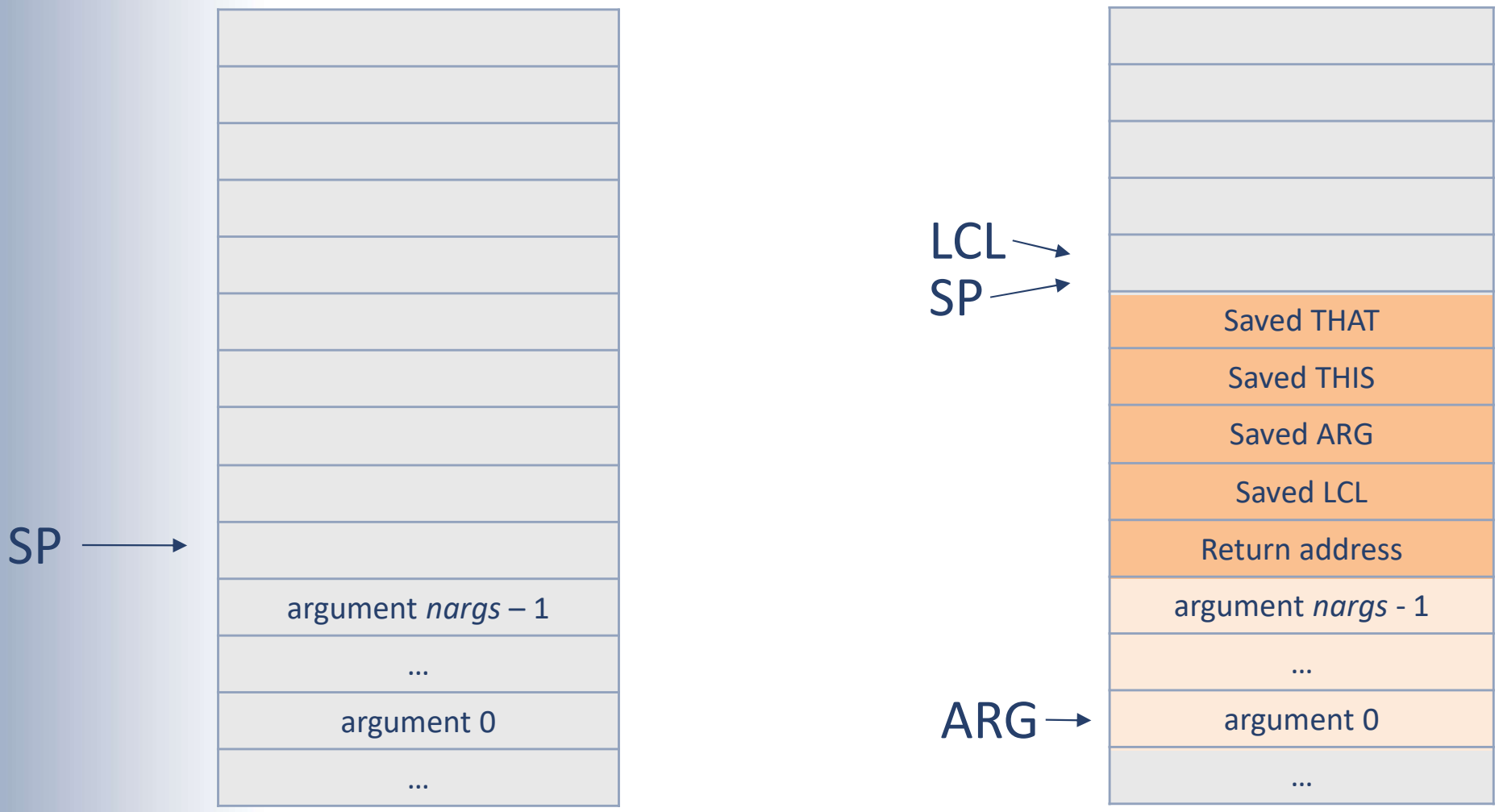
Implementing `call`

call *name nargs*

Output assembly code to create bookkeeping data, setup ARG and LCL pointers, and jump to start of function:

1. Push the return address (use label declared below)
2. Push LCL
3. Push ARG
4. Push THIS
5. Push THAT
6. Set ARG = SP – *nargs* – 5
7. Set LCL = SP
8. Goto *name*
9. Declare return address label (must be globally unique!)

Stack (`call`)



call *name nargs*

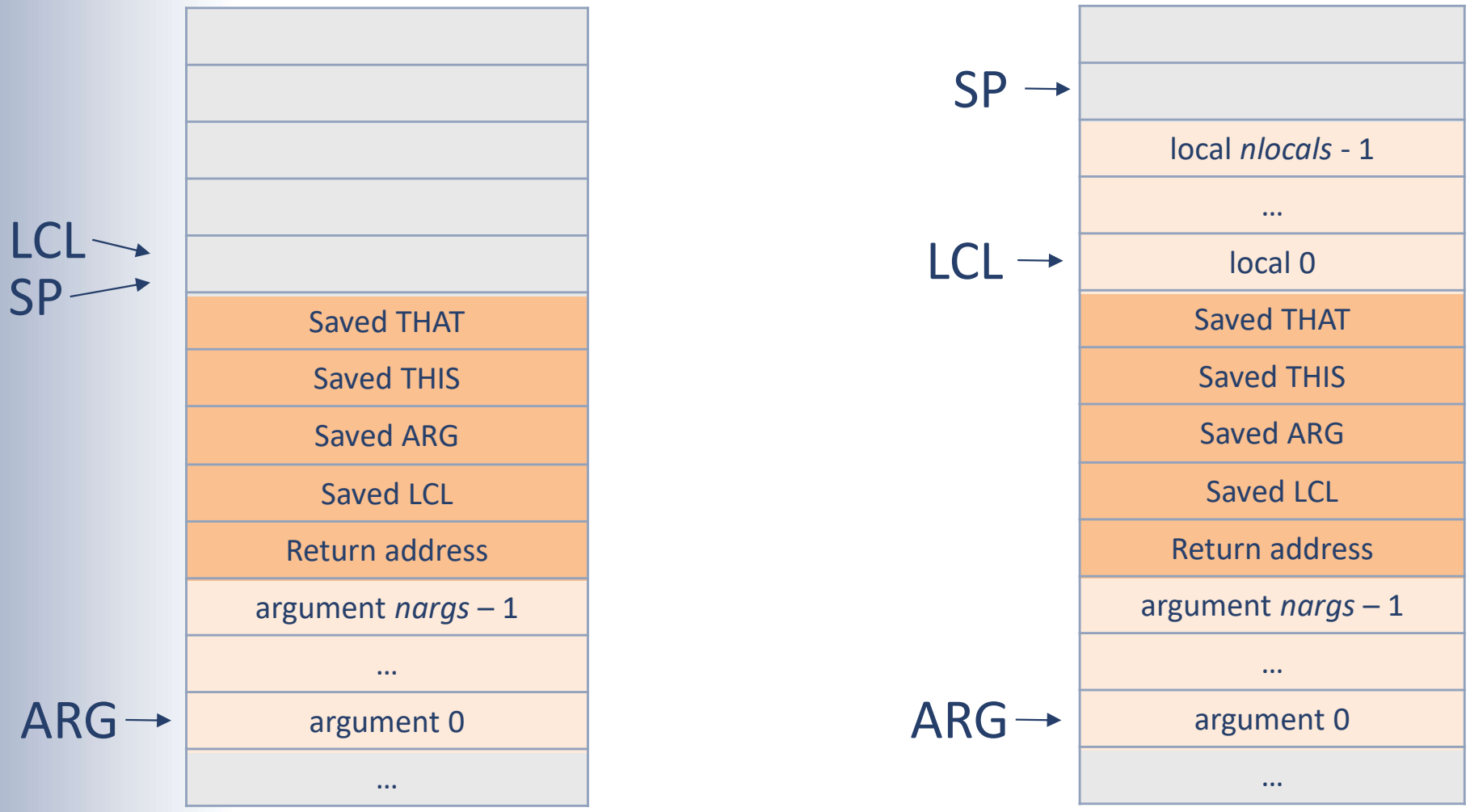
Implementing `function`

function *name nlocals*

Do (in assembly):

1. Set label for function name
2. Push 0 *nlocals* times (creating `local` segment)

Stack (function)



(before)

function *name nlocals*

(after)

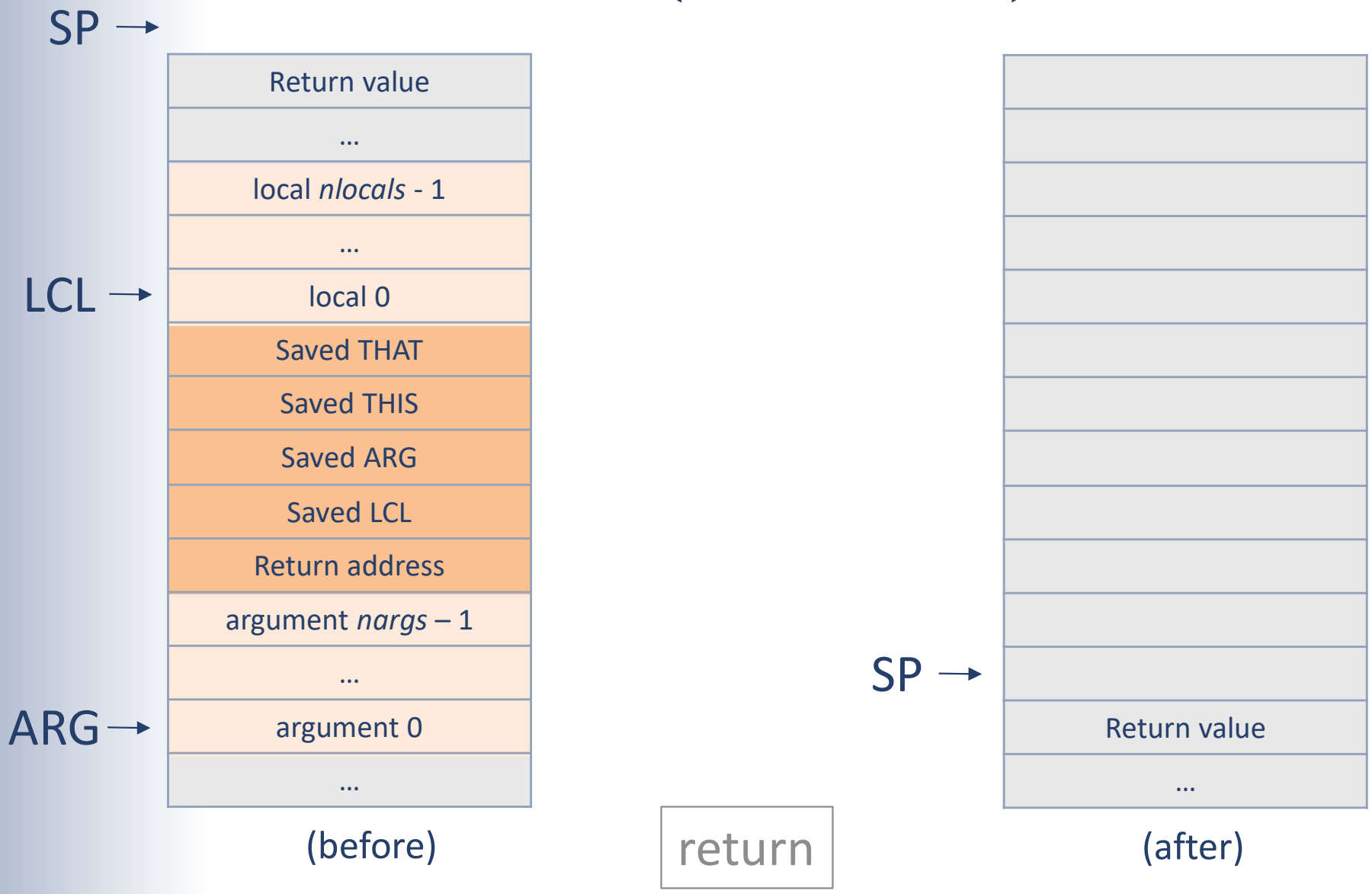
Implementing `return`

`return`

This is the most difficult of the three! Do:

1. Store address in LCL to a temp location (e.g., in R13) – call this FRAME
2. Store return address (currently stored in FRAME – 5) in another temp location (e.g., R14) – call this RET
3. Copy return value to location held in ARG
4. Set SP = ARG + 1 (effectively, pop entire stack frame)
5. Restore pointers:
   - THAT ← address stored in FRAME – 1
   - THIS ← address stored in FRAME – 2
   - ARG ← address stored in FRAME – 3
   - LCL ← address stored in FRAME – 4
6. Jump unconditionally to RET

Stack (return)



PUTTING IT ALL TOGETHER

Bootstrap Code

Hack program always starts at ROM[0]:

What do we put there?

Assembly code to:

- SP = 256 // set start location of stack
- call Sys.init // VM defined start function
  // (supplied by programmer)

VM Translator (Ultimate)

Run: VMTranslator *directory*

- Directory contains all .vm files for a program
- One file should be Sys.vm, containing the Sys.init function

VMTranslator does the following:

- Creates *directory*.asm
- Emits bootstrap code into *directory*.asm
- Translates each .vm file (appending to *directory*.asm)

Eating the Project 8 Elephant

- Provided to you: 5 test "programs"
  - Get ProgramFlow programs working first (goto, etc.)
  - Then do FunctionCalls programs (one at a time!)
- ProgramFlow, first FunctionCalls programs *do not* include Sys.init
  - Translate single .vm file to .asm
  - Test using .tst script in CPUEmulator (test script does initialization for you)
- Last 2 are multi-file (including Sys.vm)
- Take your time!

Hints & Tips

- Include comments in emitted assembly code
  - Write a function for this in your translator, use it e.g., to note start of VM commands
  - Why? You will likely need to debug in part by staring at your assembly code.
- When you fail a test:
  - Figure out what the test code is doing
  - Figure out what the .vm code is supposed to do (you have the nand2tetris VMEmulator for this purpose)
  - Figure out what your code was expected to do
  - Compare your output to expected output to see where expectations differed from reality
  - Together with incremental development, this should help you narrow down your search to a single VM command

Up Next

- Thursday 100% workday for project 7 & 8
- After spring break
  - We learn the Jack language and write a compiler.